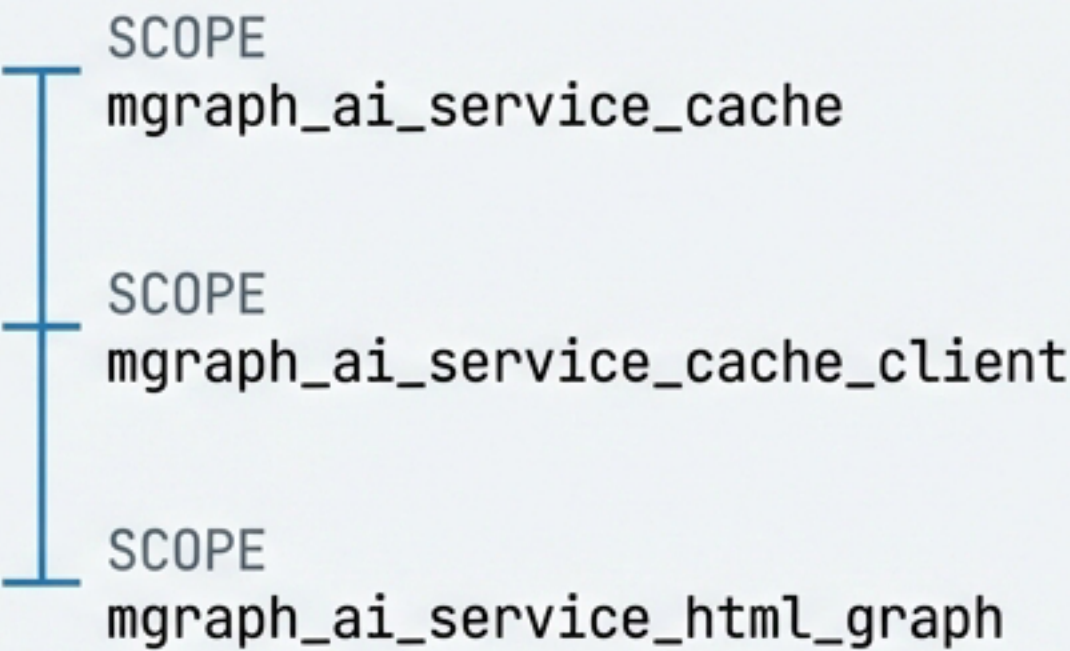


In-Memory Service Pattern for FastAPI Microservices



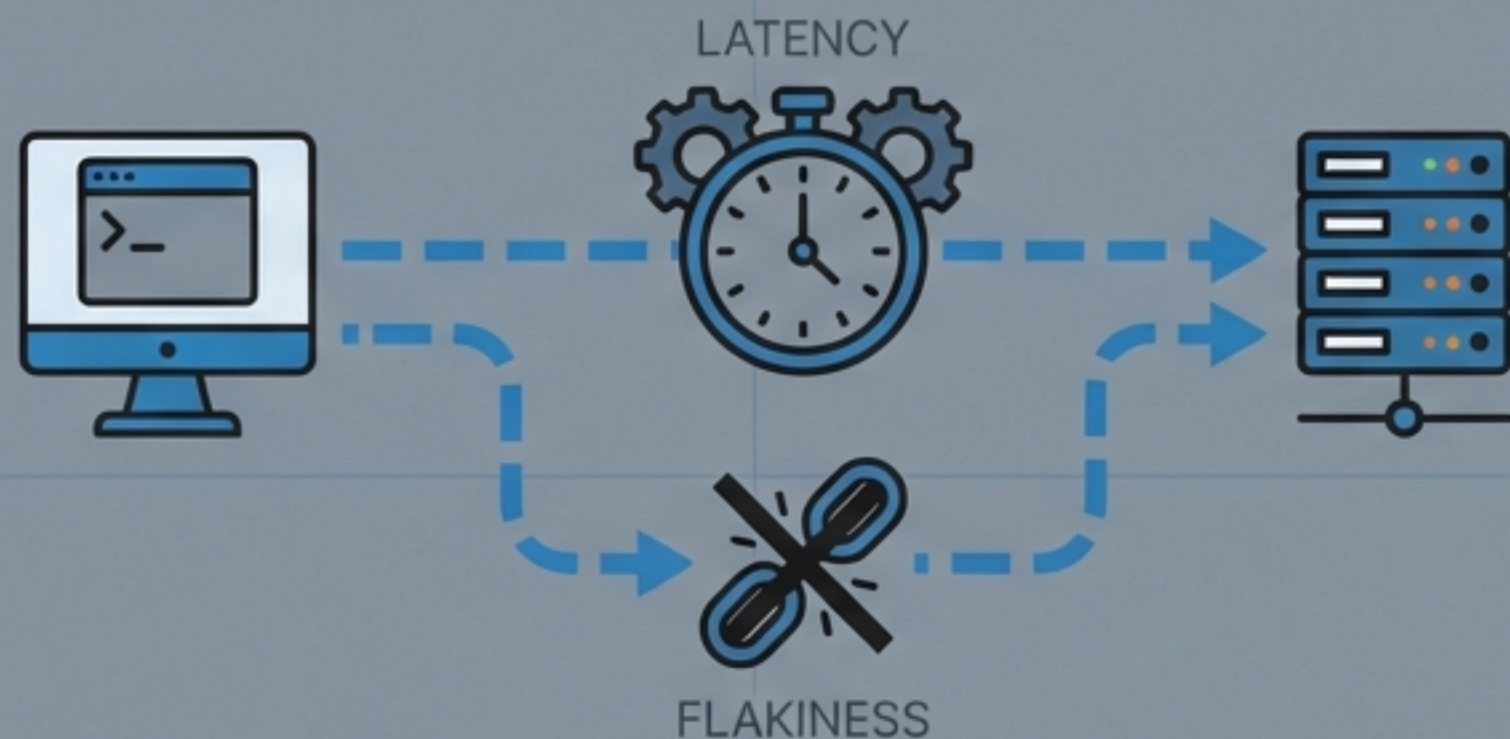
VERSION 1.5.0 | JANUARY 2025

Achieving Zero-Compromise Flexibility in Testing and Production.



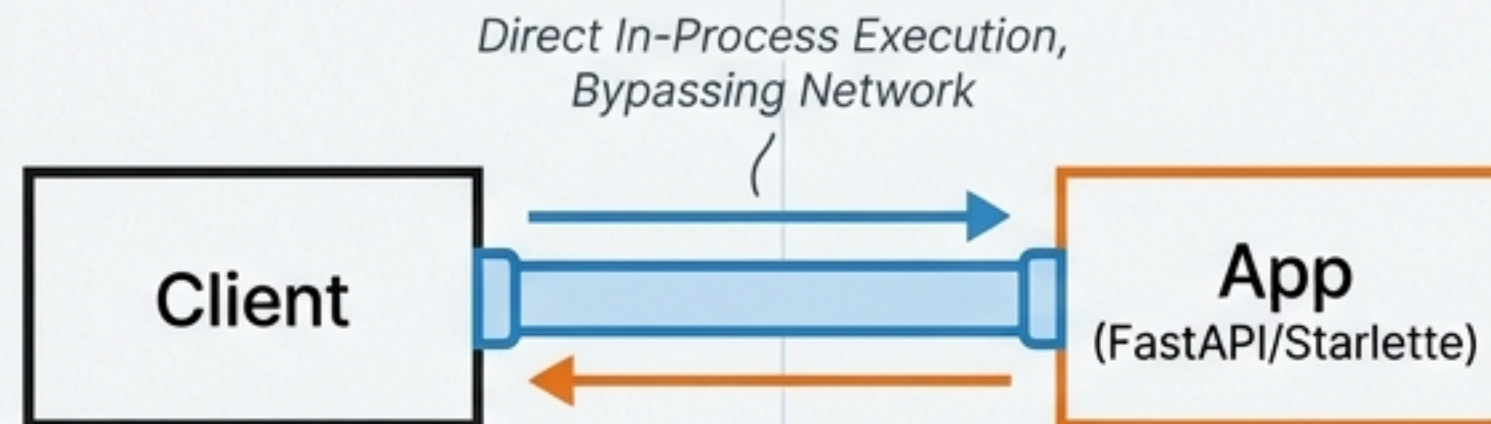
OPERATIONAL MODES
01. Full Remote // Production-grade HTTP communication
02. Full In-Memory // Millisecond-latency testing via `TestClient`
03. Hybrid // Flexible composition for local development

Traditional Microservice Testing



- Network Latency overheads
- Complex Mocking of HTTP Servers
- Flaky Integration Tests

The ASGI Breakthrough



Starlette's `TestClient` executes FastAPI apps directly in-process.

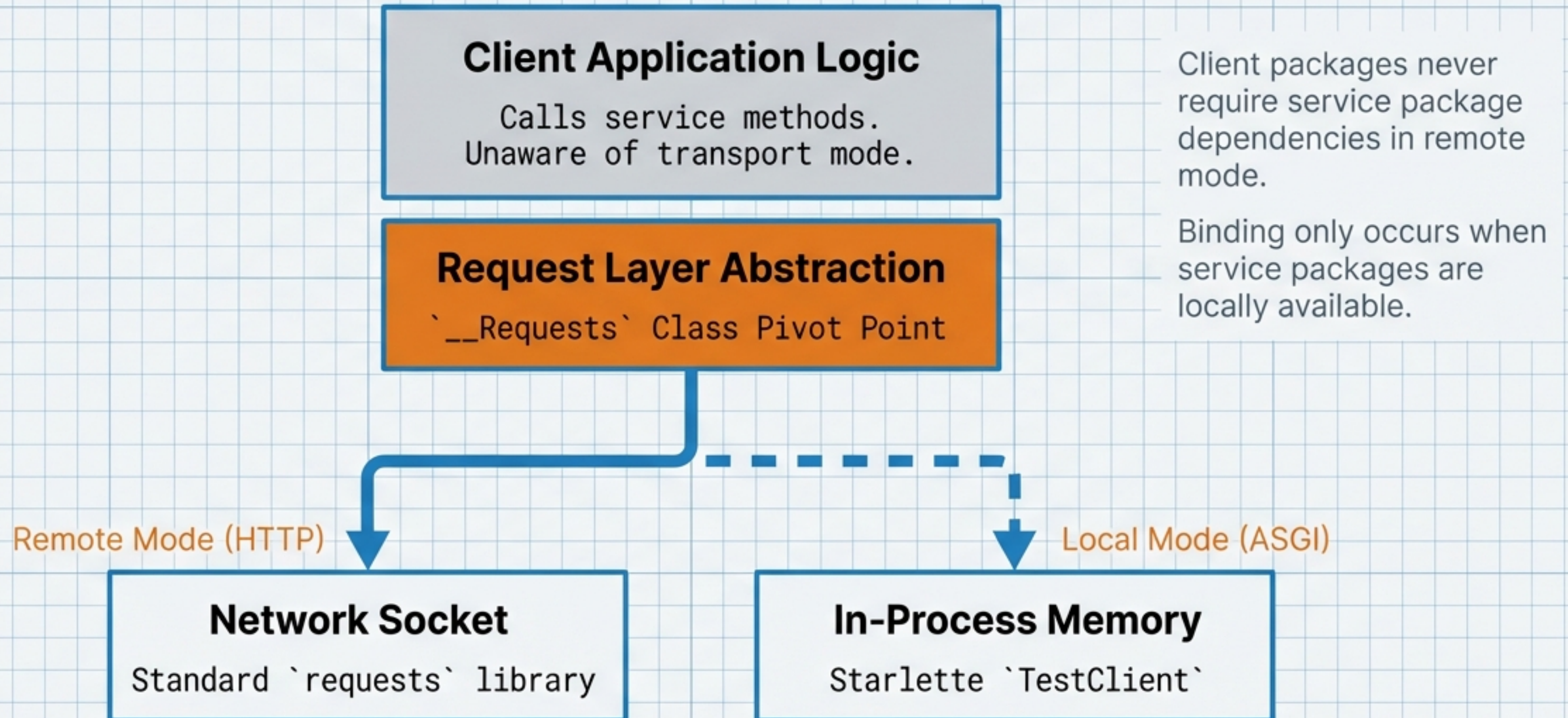
The Interface is Identical.

```
response = requests.get("/health") # Remote
response = client.get("/health")    # In-Memory
```

} Client logic remains agnostic.

High-Level Architecture: The Switch

Dependency injection occurs strictly at the request layer.



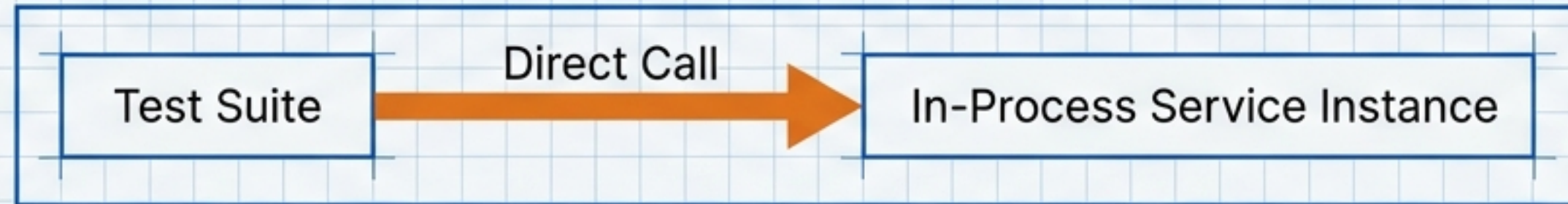
The Three Composition Scenarios

Full Remote (Production)



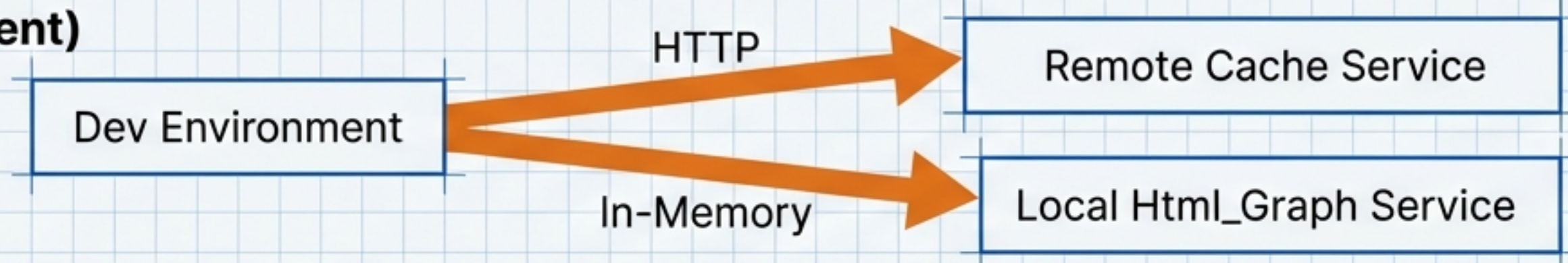
Pure client packages. No service dependencies.

Full In-Memory (Testing)



Millisecond execution. Zero flakiness.

Hybrid (Development)

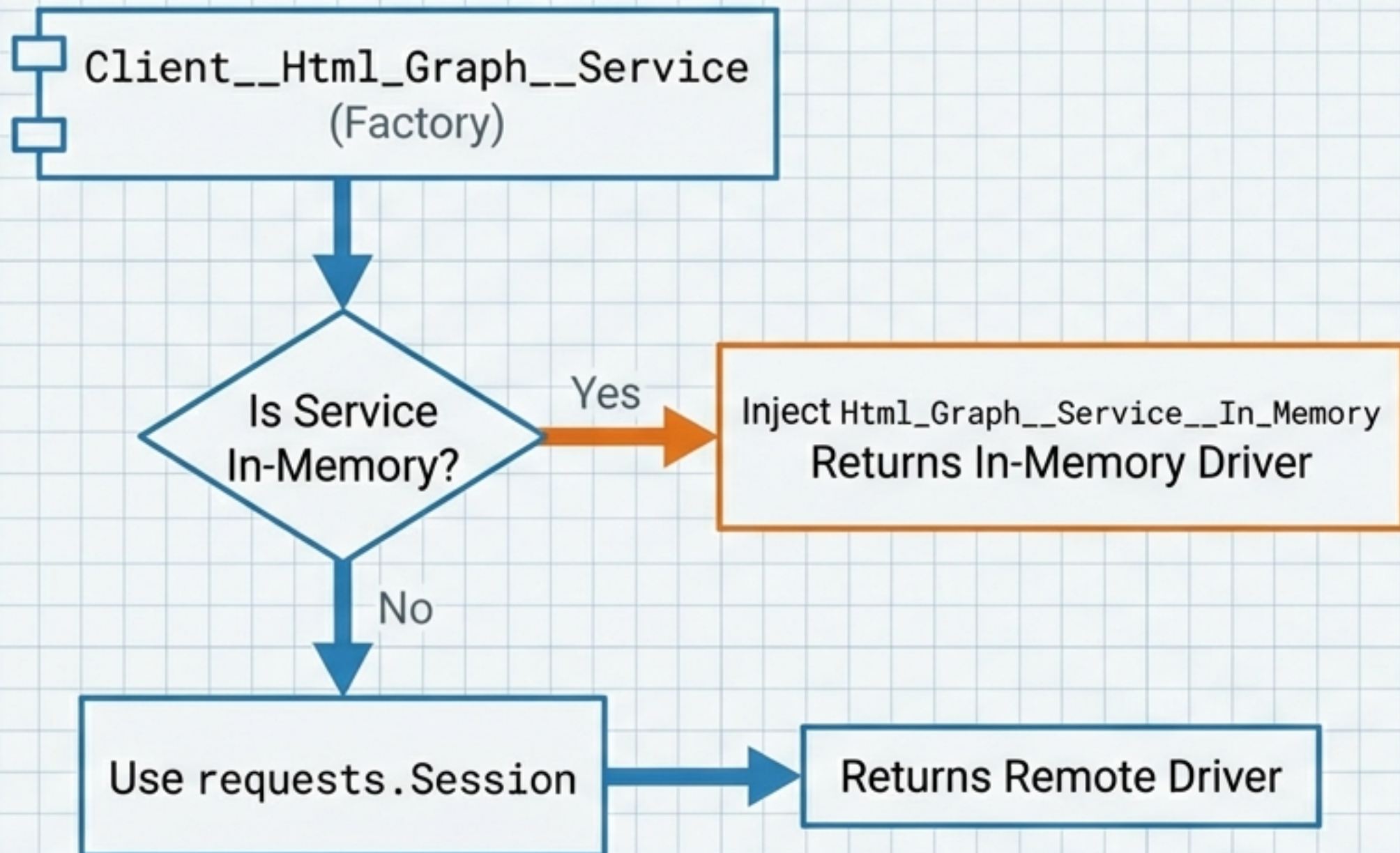


Flexible development targets. Mix and match.

The Pivot Point: Refactoring the Client

Act II: The Specifications

The Factory Logic Visualization



Single Injection Point

This Factory Wrapper is the **only* injection point. The rest of the application remains unaware of the mode switch.

``mgraph_ai_service_html_graph``
package

Spec 1: The Request Abstraction

Standardising the Transport Layer

The Interface Adapter

```
class Html_Graph__Service__Client__Requests
```

Input:

```
config: Html_Graph__Service__Client__Config
```

Internal Logic:

Wraps transport mechanism (hides `Session` vs `TestClient` intricacies).

Methods (Standardised Signatures):

```
.get()  
.post()  
.put()
```

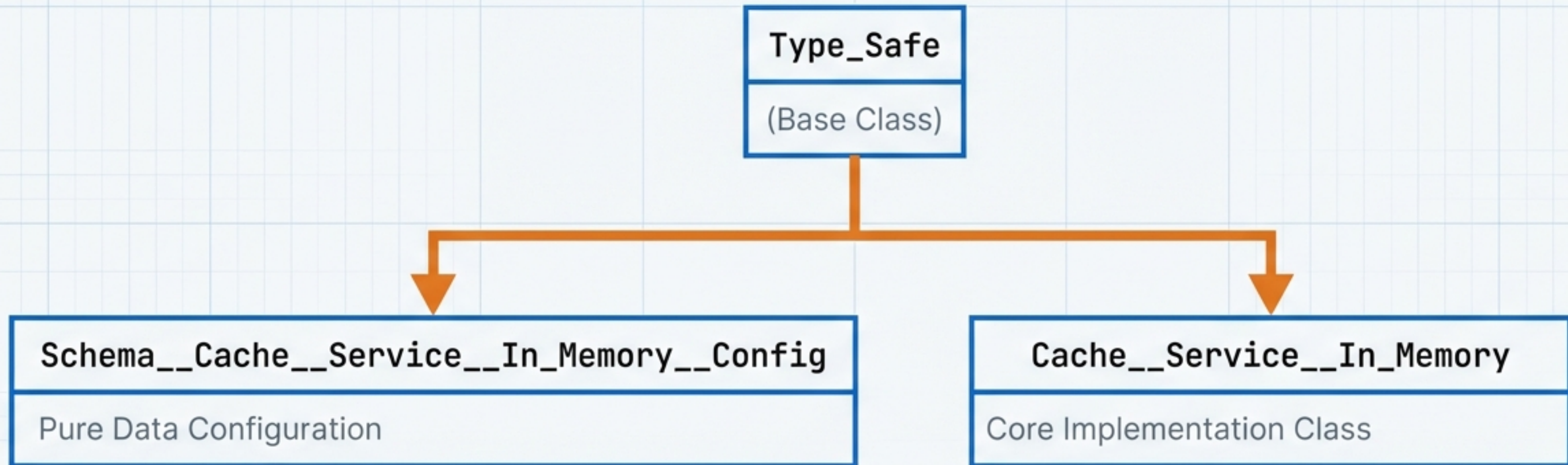
Output:

```
Schema__Html_Graph__Client__Requests__Result
```

Upper layers call these methods without managing connection state or transport type.

Spec 2: The Cache Service (Foundation)

Package: `mgraph\_ai\_service\_cache`

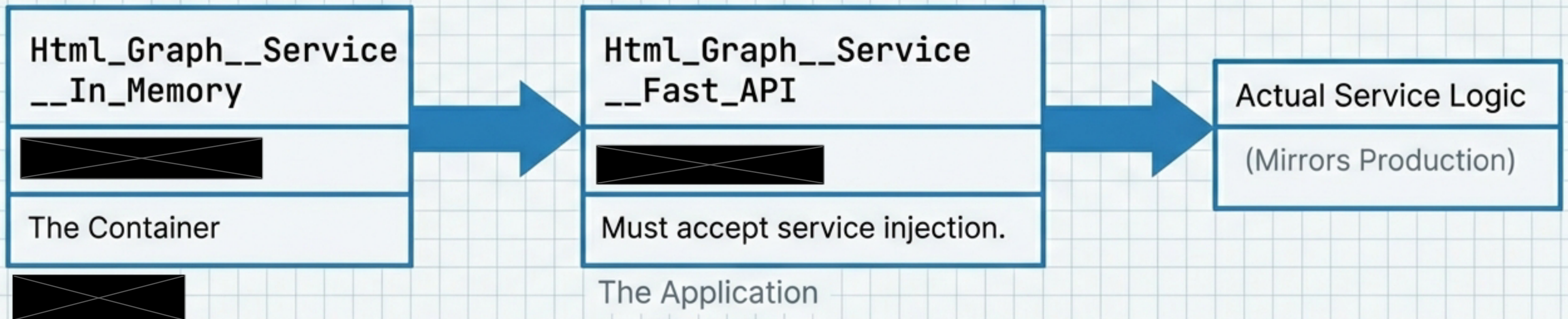


This represents the simpler implementation of the pattern. It serves as the Proof of Concept (Phase 1) before tackling the complex Graph service.

Spec 3: The HTML Graph Service (Complexity)

Package: `mgraph\_ai\_service\_html\_graph`

The **`Html\_Graph\_Service\_Fast\_API`** class must exist and be instantiable within the client process to function in this mode.



Configuration & Schema Design Strategy

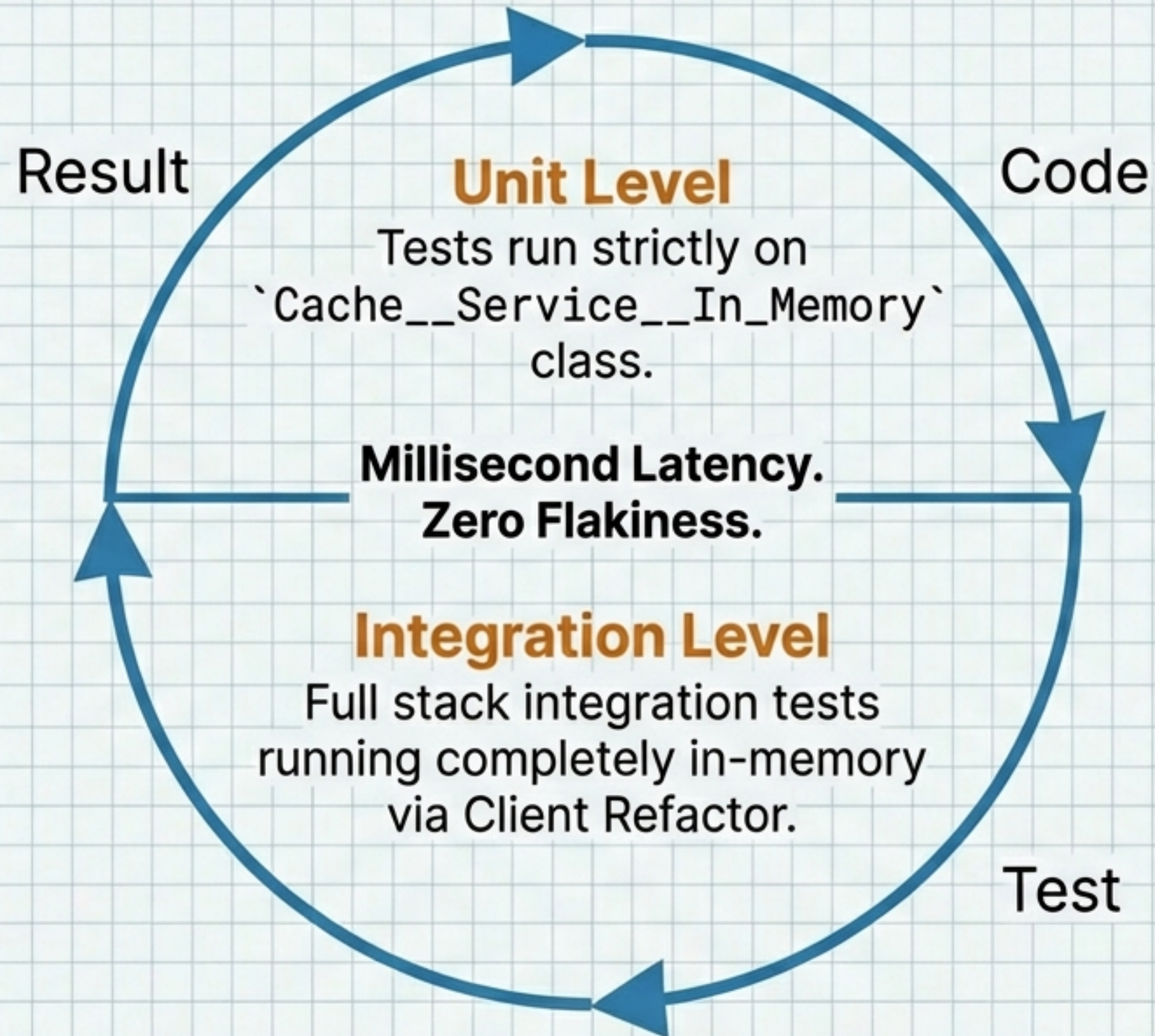
- 1. Pure Data:** Schemas contain no methods (except `from_json`).
- 2. Type Safety:** All attributes must be typed. No raw primitives.
- 3. Inheritance:** All classes must inherit from `Type_Safe`.

```
class Schema__Cache__Service__In_Memory__Config(Type_Safe):  
    data_path: Safe_Str  
    max_size_mb: Safe_Int  
    ttl_seconds: Safe_Int  
    persist_on_exit: Safe_Bool
```



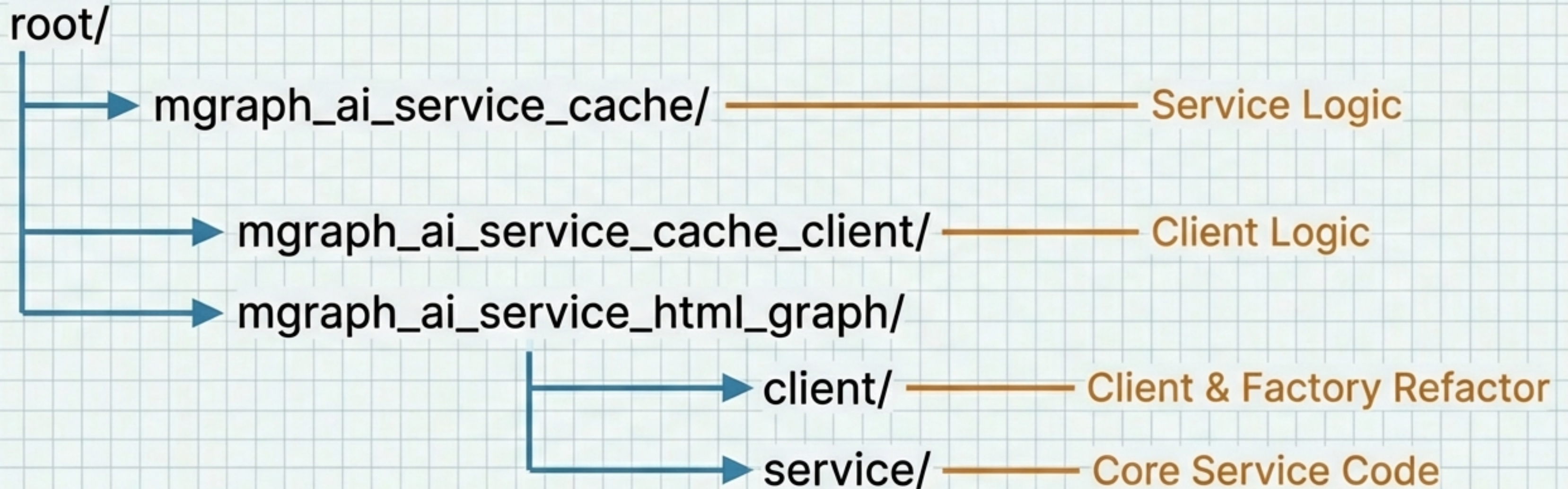
Strict Wrapper Types

Accelerated Testing Patterns



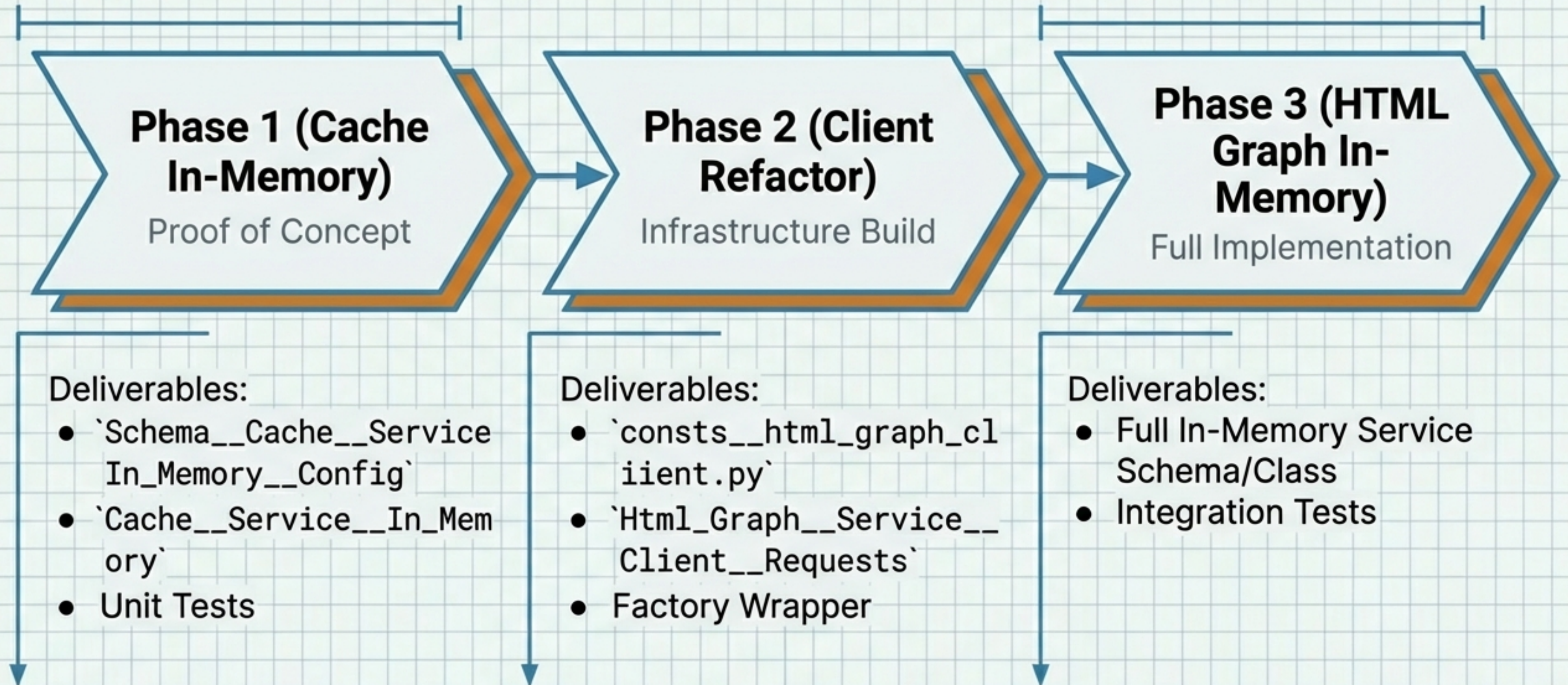
File Structure & Organisation

Act III: Execution & Standards



Modular separation ensures production builds do not accidentally import test-logic dependencies.

Implementation Roadmap



Pre-Implementation Checklist



Confirm Service Existence:

Verify `Html_Graph__Service` class exists and check constructor signature.



Confirm API Layer:

Ensure `Html_Graph__Service__Fast_API` exists and accepts service injection.



Verify Client Pattern:

Check `Client__Cache__Service` pattern for `set__fast_api_app().client()`.



Environment Audit:

Check variable naming conventions in existing cache client code to avoid conflicts.

Code Quality & Standards



Type Safety

All classes must inherit from **Type_Safe**.
All attributes must be strictly typed.



Data Integrity

No raw `str`, `int`, or `float`.
Use **Safe_*** wrapper types.



Formatting

Visual alignment must follow the Python Formatting Guide.
Inline comments only (no docstrings).



Fluent Interface

`setup()` methods must return `self` to enable method chaining.

The Zero-Compromise Promise



TESTING

Full in-memory stack means instant feedback.



DEVELOPMENT

Hybrid modes allow targeted debugging.



PRODUCTION

Zero dependency bloat.

The ``__Requests`` class is the only injection point. Everything above it is mode-agnostic.